

# JAVA

## COMPLETE NOTES

*Easy Language • Simple Examples • Important Topics Highlighted*

★ Red star = Important Topic (read carefully, exam / interview favorite)

**MMOC TEACH**  
**MADE BY PRAVENDRA**

# INDEX

- ★ **1. Introduction to Java**
- ★ **2. Variables and Data Types**
  - 3. Type Casting
- ★ **4. Operators**
  - 5. Taking Input using Scanner
- ★ **6. Conditional Statements (if, else if, else)**
- ★ **7. Loops - for, while, do-while**
  - 8. Break and Continue
- ★ **9. Arrays**
- ★ **10. Strings**
- ★ **11. Methods (Functions)**
  - 12. Method Overloading
- ★ **13. Constructors**
- ★ **14. OOP - Classes and Objects**
- ★ **15. Inheritance**
- ★ **16. Polymorphism**
  - 17. Encapsulation
  - 18. Abstraction (Abstract Class and Interface)
  - 19. Static Keyword
- ★ **20. Exception Handling (try, catch, finally)**
- ★ **21. Collections - ArrayList**
  - 22. Collections - HashMap
  - 23. Wrapper Classes and Autoboxing
  - 24. Access Modifiers
  - 25. The 'final' Keyword

# ★ 1. Introduction to Java [IMPORTANT TOPIC]

---

## Explanation:

Java is a very popular programming language used to build apps, websites, and large business software. Java code first gets converted into 'bytecode', which can then run on ANY computer or phone that has Java installed. This is why Java's motto is 'Write Once, Run Anywhere'.

Java is an Object-Oriented language, which means it organizes code using classes and objects, similar to how we think about real-world things.

## Key Points:

- Platform independent (runs on Windows, Mac, Linux, Android, etc.).
- Object-Oriented programming language.
- Strongly typed — every variable must have a fixed data type.
- Very popular for Android apps and large company software.

## Example Code:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

### Output:

Hello, World!

💡 *Note: Every Java program must have a main() method — this is where the program starts running.*

## ★ 2. Variables and Data Types [IMPORTANT TOPIC]

---

### Explanation:

A variable is a container that stores a value. In Java, we must tell the computer the data type of the variable before using it — this is called 'strongly typed'.

### Key Points:

- int -> whole numbers, example: 10
- double -> decimal numbers, example: 3.14
- char -> a single character, example: 'A'
- String -> text, example: "Hello"
- boolean -> true or false

### Example Code:

```
String name = "Pravendra";  
int age = 21;  
double height = 5.9;  
boolean isStudent = true;  
  
System.out.println(name + " is " + age + " years old");
```

### Output:

```
Pravendra is 21 years old
```

### 3. Type Casting

---

#### Explanation:

Type casting means converting one data type into another. 'Widening' casting (small to big type) happens automatically. 'Narrowing' casting (big to small type) must be done manually.

#### Example Code:

```
double price = 99.99;
int roundedPrice = (int) price;    // narrowing - manual casting
System.out.println(roundedPrice);

int num = 10;
double bigNum = num;               // widening - automatic
System.out.println(bigNum);
```

#### Output:

```
99
10.0
```

## ★ 4. Operators [IMPORTANT TOPIC]

---

### Explanation:

Operators are symbols used to perform actions on values, like adding two numbers or comparing which one is bigger.

### Key Points:

- Arithmetic: + - \* / %
- Comparison: == != > < >= <=
- Logical: && (and), || (or), ! (not)
- Assignment: = += -= \*= /=

### Example Code:

```
int a = 10, b = 3;  
System.out.println(a + b);      // 13  
System.out.println(a % b);      // 1 (remainder)  
System.out.println(a > b && b > 0); // true
```

### Output:

```
13  
1  
true
```

## 5. Taking Input using Scanner

---

### Explanation:

The Scanner class is used to take input from the user through the keyboard. We must first import it before using it.

### Example Code:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        System.out.println("Hello, " + name);
    }
}
```

### Output:

```
Enter your name: Pravendra
Hello, Pravendra
```

 *Note: Use `sc.nextInt()` for numbers, `sc.nextLine()` for full text lines.*

## ★ 6. Conditional Statements (if, else if, else) [IMPORTANT TOPIC]

---

### Explanation:

Conditional statements help the program make decisions. The code inside 'if' runs only when its condition is true. 'else if' checks another condition, and 'else' runs when nothing above is true.

### Example Code:

```
int marks = 75;

if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 60) {
    System.out.println("Grade B");
} else {
    System.out.println("Grade C");
}
```

### Output:

Grade B

## ★ 7. Loops - for, while, do-while [IMPORTANT TOPIC]

---

### Explanation:

A loop repeats a block of code many times. A 'for' loop is used when we know how many times to repeat. A 'while' loop repeats as long as a condition is true. A 'do-while' loop always runs at least once, even if the condition is false.

### Example Code:

```
for (int i = 1; i <= 3; i++) {  
    System.out.println("For loop: " + i);  
}  
  
int n = 1;  
while (n <= 3) {  
    System.out.println("While loop: " + n);  
    n++;  
}
```

### Output:

```
For loop: 1  
For loop: 2  
For loop: 3  
While loop: 1  
While loop: 2  
While loop: 3
```

## 8. Break and Continue

---

### Explanation:

'break' stops the loop completely and exits it. 'continue' skips only the current step and moves on to the next round of the loop.

### Example Code:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;    // skip 3  
    if (i == 5) break;      // stop at 5  
    System.out.println(i);  
}
```

### Output:

```
1  
2  
4
```

## ★ 9. Arrays [IMPORTANT TOPIC]

---

### Explanation:

An array is used to store multiple values of the SAME data type together, under one variable name. The size of an array is fixed once it is created.

### Example Code:

```
int[] numbers = {10, 20, 30, 40};

System.out.println(numbers[0]);    // first item
System.out.println(numbers.length);

for (int n : numbers) {
    System.out.println(n);
}
```

### Output:

```
10
4
10
20
30
40
```

💡 Note: Array index always starts from 0, not 1.

## ★ 10. Strings [IMPORTANT TOPIC]

---

### Explanation:

A String is used to store text. In Java, String is not a basic data type — it is actually a class, and it comes with many useful built-in methods.

### Key Points:

- `length()` -> number of characters
- `toUpperCase()` / `toLowerCase()` -> change case
- `trim()` -> remove extra spaces
- `substring(0,3)` -> get part of the text
- `equals()` -> compare two strings (never use `==` for text comparison)

### Example Code:

```
String text = " Hello Java ";  
System.out.println(text.trim());  
System.out.println(text.trim().toUpperCase());  
System.out.println(text.trim().length());
```

#### Output:

```
Hello Java  
HELLO JAVA  
10
```

 *Note: Always use `.equals()` to compare two String values, not `==`.*

## ★ 11. Methods (Functions) [IMPORTANT TOPIC]

---

### Explanation:

A method is a reusable block of code that performs one task. Instead of writing the same code again and again, we write it once inside a method and call it whenever needed.

### Example Code:

```
public static int addNumbers(int a, int b) {  
    return a + b;  
}  
  
public static void main(String[] args) {  
    int total = addNumbers(5, 7);  
    System.out.println("Total: " + total);  
}
```

### Output:

Total: 12

## 12. Method Overloading

---

### Explanation:

Method overloading means having multiple methods with the SAME name but different parameters (either different number or different type of inputs). Java automatically chooses the correct one based on what you pass.

### Example Code:

```
static int add(int a, int b) {  
    return a + b;  
}  
  
static double add(double a, double b) {  
    return a + b;  
}  
  
System.out.println(add(2, 3));           // uses int version  
System.out.println(add(2.5, 3.5));       // uses double version
```

### Output:

```
5  
6.0
```

## ★ 13. Constructors [IMPORTANT TOPIC]

---

### Explanation:

A constructor is a special method that runs automatically when a new object is created from a class. It is mainly used to set the starting values of an object. A constructor has the SAME name as the class and no return type.

### Example Code:

```
class Student {
    String name;
    int marks;

    // Constructor
    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
}

Student s1 = new Student("Pravendra", 90);
System.out.println(s1.name + " scored " + s1.marks);
```

### Output:

Pravendra scored 90

 *Note: 'this' refers to the current object — it is used to separate the class variable from the parameter with the same name.*

## ★ 14. OOP - Classes and Objects [IMPORTANT TOPIC]

---

### Explanation:

OOP (Object-Oriented Programming) is a way of writing programs using 'objects' that represent real-world things, like a Car, a Student, or a Bank Account.

A class is a blueprint/design for creating objects. An object is a real item created using that blueprint (using the 'new' keyword).

### Example Code:

```
class Car {  
    String brand;  
  
    void showBrand() {  
        System.out.println("Brand: " + brand);  
    }  
}  
  
Car c1 = new Car();  
c1.brand = "Toyota";  
c1.showBrand();
```

### Output:

Brand: Toyota

## ★ 15. Inheritance [IMPORTANT TOPIC]

---

### Explanation:

Inheritance allows one class (child/subclass) to use the properties and methods of another class (parent/superclass), using the keyword 'extends'. This helps us reuse code instead of writing it again and again.

### Example Code:

```
class Animal {  
    void eat() {  
        System.out.println("This animal eats food");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("The dog barks");  
    }  
}  
  
Dog d = new Dog();  
d.eat();    // inherited from Animal  
d.bark();   // defined in Dog
```

### Output:

```
This animal eats food  
The dog barks
```

## ★ 16. Polymorphism [IMPORTANT TOPIC]

---

### Explanation:

Polymorphism means 'many forms'. It allows the same method name to behave differently, either through method overriding (child class changes the parent's method) or method overloading (same method, different parameters).

### Example Code:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat says Meow");
    }
}

Animal a = new Cat();
a.sound();    // calls Cat's version, not Animal's
```

### Output:

Cat says Meow



*Note: @Override is used above a method to show it is replacing (overriding) the parent class's method.*

## 17. Encapsulation

---

### Explanation:

Encapsulation means hiding the internal data of a class by making variables 'private', and allowing access to them only through public methods, usually called getters and setters. This protects the data from being changed incorrectly.

### Example Code:

```
class BankAccount {  
    private double balance;  
  
    public void setBalance(double amount) {  
        if (amount >= 0) balance = amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}  
  
BankAccount acc = new BankAccount();  
acc.setBalance(5000);  
System.out.println(acc.getBalance());
```

### Output:

5000.0

## 18. Abstraction (Abstract Class and Interface)

---

### Explanation:

Abstraction means showing only the important details and hiding the complicated working. In Java, this is done using 'abstract classes' (which can have some finished and some unfinished methods) or 'interfaces' (which only describe what methods a class MUST have).

### Example Code:

```
abstract class Shape {  
    abstract void draw();    // no body - must be completed by child class  
}  
  
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing a circle");  
    }  
}  
  
Shape s = new Circle();  
s.draw();
```

### Output:

Drawing a circle

## 19. Static Keyword

---

### Explanation:

'static' means a variable or method belongs to the CLASS itself, not to any single object. This means it can be used directly without creating an object first, and it is shared by all objects of that class.

### Example Code:

```
class Counter {  
    static int count = 0;  
  
    Counter() {  
        count++;  
    }  
}  
  
new Counter();  
new Counter();  
new Counter();  
System.out.println(Counter.count);
```

### Output:

3

 *Note: main() is always static, because Java calls it directly without creating an object first.*

## ★ 20. Exception Handling (try, catch, finally) [IMPORTANT TOPIC]

---

### Explanation:

Sometimes a program crashes because of an unexpected error (called an exception), like dividing by zero. Exception handling lets us 'catch' this error safely and keep the program running, instead of it stopping suddenly.

### Example Code:

```
try {
    int result = 10 / 0;
    System.out.println(result);
} catch (ArithmeticException e) {
    System.out.println("Error: Cannot divide by zero!");
} finally {
    System.out.println("Program finished running.");
}
```

### Output:

```
Error: Cannot divide by zero!
Program finished running.
```

 *Note: The 'finally' block always runs, whether there is an error or not.*

## ★ 21. Collections - ArrayList *[IMPORTANT TOPIC]*

---

### Explanation:

A normal array has a fixed size, but an ArrayList can grow or shrink automatically. It is part of Java's Collection framework and is one of the most used data structures in real Java programs.

### Example Code:

```
import java.util.ArrayList;

ArrayList<String> fruits = new ArrayList<>();
fruits.add("Apple");
fruits.add("Banana");
fruits.add("Mango");
fruits.remove("Banana");

System.out.println(fruits);
System.out.println(fruits.get(0));
System.out.println(fruits.size());
```

### Output:

```
[Apple, Mango]
Apple
2
```

## 22. Collections - HashMap

---

### Explanation:

A HashMap stores data in 'key : value' pairs, similar to a real dictionary. It is very useful for storing related information where each value can be quickly found using its key.

### Example Code:

```
import java.util.HashMap;

HashMap<String, Integer> marks = new HashMap<>();
marks.put("Pravendra", 90);
marks.put("Aman", 85);

System.out.println(marks.get("Pravendra"));
System.out.println(marks);
```

### Output:

```
90
{Pravendra=90, Aman=85}
```

## 23. Wrapper Classes and Autoboxing

---

### Explanation:

A wrapper class turns a basic (primitive) data type like int or double into a proper object, like Integer or Double. This is needed because some parts of Java, like ArrayList, only work with objects, not basic types. Java can also convert between them automatically, called 'autoboxing'.

### Example Code:

```
int a = 10;  
Integer obj = a;      // autoboxing - int to Integer, automatic  
int b = obj;          // unboxing - Integer to int, automatic  
  
System.out.println(obj);  
System.out.println(b);
```

### Output:

```
10  
10
```

## 24. Access Modifiers

---

### Explanation:

Access modifiers control WHO can use a variable, method, or class from other parts of the program. They help protect data and keep code organized and secure.

### Key Points:

- public -> can be accessed from anywhere
- private -> can only be accessed inside the same class
- protected -> can be accessed in the same package and by child classes
- default (nothing written) -> can be accessed only within the same package

### Example Code:

```
class Student {  
    private String name;    // only accessible inside this class  
    public int age;         // accessible from anywhere  
}
```

### Output:

(No console output - this shows how access modifiers control visibility of class members.)

## 25. The 'final' Keyword

---

### Explanation:

'final' is used when we want something to stay fixed and NOT be changed later. A final variable becomes a constant, a final method cannot be overridden, and a final class cannot be inherited by another class.

### Example Code:

```
final double PI = 3.14159;  
System.out.println(PI);  
// PI = 3.2;    // This will give an ERROR - final variables cannot be  
changed
```

### Output:

```
3.14159
```